

ZORQEN RESEARCH

Master Product, Architecture, Research and Delivery Specification

Project definition

A separate autonomous strategy-research and qualification platform. It discovers and improves controlled strategy candidates, validates them against immutable data and policy gates, and transfers only qualified candidate packages to MOMO Quant (later Zorqen Quant) for independent verification, paper forward testing, and human-controlled deployment.

Document	Value
Version	0.1 - Initial implementation baseline
Status	Approved direction; implementation remains milestone-controlled
Date	3 August 2026
Initial environment	Windows laptop during development; Windows/Linux VPS later
Initial strategy families	Adaptive Multi-Timeframe Trend Breakout (primary); Support and Resistance (secondary)
Authority boundary	Research and qualification only; no direct paper/live trading authority

Document rule: This PDF is the single product specification. Repository files `AGENTS.md` and `PROJECT_STATUS.md` govern permanent coding-agent behavior and current implementation status.

Table of Contents

1. Document Control and Interpretation	6
1.1 Purpose	6
1.2 Precedence	6
1.3 Core terminology	6
2. Executive Summary	7
2.1 Initial strategy scope	7
2.2 Fundamental separation of duties	7
3. Vision, Mission and Product Principles	8
3.1 Vision	8
3.2 Mission	8
3.3 Product principles	8
4. Scope, Boundaries and Non-Goals	9
4.1 In scope	9
4.2 Explicit MVP non-goals	9
4.3 Scope-control rule	9
5. Roles and Operating Model	10
5.1 Initial roles	10
5.2 Human workflow	10
5.3 Bounded autonomy	10
6. Initial Strategy Families	11
6.1 Shared requirements	11
6.2 Adaptive Multi-Timeframe Trend Breakout - primary	11
Permitted early research dimensions	11
Protected semantics	11
6.3 Support and Resistance - secondary	11
Permitted early research dimensions	11
7. Controlled Strategy Representation	12
7.1 Why unrestricted code generation is deferred	12
7.2 Definition layers	12
7.3 Illustrative definition	12
7.4 Cross-engine parity	12
8. Autonomous Research Loop	13
8.1 Campaign setup	13
8.2 Experiment loop	13

8.3 Keep/discard policy	13
8.4 Search methods	13
9. Data Governance and Leakage Prevention	14
9.1 Canonical market data	14
9.2 Dataset snapshots	14
9.3 Temporal isolation	14
9.4 Data-quality gates	14
10. Backtesting and Validation Methodology	15
10.1 Simulation model	15
10.2 Required realism	15
10.3 Validation stages	15
10.4 Reproducibility	15
11. Qualification Policy: Hard Gates and Score	16
11.1 Hard rejection gates	16
11.2 Provisional weighted score	16
11.3 Initial qualification policy	16
11.4 Provisional thresholds	16
12. Candidate Lifecycle	17
12.1 Authority limits	17
12.2 Immutability	17
12.3 Rejection as a first-class result	17
13. Candidate Package and MOMO Quant Integration	18
13.1 Integration sequence	18
13.2 Required contents	18
13.3 Illustrative envelope	18
13.4 MOMO Quant responsibilities	18
14. System Architecture	19
14.1 Major components	19
14.2 Initial deployment shape	19
15. Recommended Technology Stack	20
15.1 CPU-first rule	20
15.2 Windows/Linux parity	20
16. Core Domain Model	21
16.1 State machines and idempotency	21
17. API and User Interface	22

17.1 Initial API groups	22
17.2 Required MVP screens	22
17.3 UX principles	22
18. Security, Isolation and Governance	23
18.1 Security boundaries	23
18.2 Governance	23
18.3 Future arbitrary-code sandbox	23
19. Observability, Failure Handling and Retention	24
19.1 Required telemetry	24
19.2 Failure handling	24
19.3 Retention	24
20. Testing and Quality Gates	25
20.1 Test layers	25
20.2 Milestone completion gates	25
21. Repository Documentation and Agent Handoff	26
21.1 Required root files	26
21.2 New-agent read order	26
21.3 Agent switching	26
22. Deployment Model	27
22.1 Local development	27
22.2 VPS deployment	27
22.3 Scaling sequence	27
23. Delivery Roadmap and Milestones	28
23.1 Milestone prompt structure	28
23.2 No milestone overlap	28
24. MVP Acceptance Criteria	29
24.1 Product acceptance	29
24.2 Technical acceptance	29
24.3 Research-integrity acceptance	29
25. Risks and Mitigations	30
25.1 Primary product risk	30
26. Decisions Frozen by Version 0.1	31
26.1 Deferred details	31
27. Initial Repository Deliverable	32
27.1 Expected foundation	32

27.2 Explicit exclusions	32
27.3 First-commit review	32
Appendix A. Development-Agent Loop	33
Appendix B. Research-Agent Loop	33
Appendix C. Glossary	33

1. Document Control and Interpretation

1.1 Purpose

This document defines what Zorqen Research is, what it is not, how its autonomous research process operates, how candidates are evaluated, how qualified candidates are transferred to MOMO Quant, and how the product will be built in controlled milestones.

It deliberately combines product vision, architecture, research governance, integration rules, delivery roadmap, testing expectations, and coding-agent handoff requirements into one source of truth.

1.2 Precedence

When sources disagree, use this order:

1. Verified repository implementation and passing tests at the authoritative commit.
2. This master specification for product intent and architectural boundaries.
3. PROJECT_STATUS.md for current verified state and handoff context.
4. AGENTS.md for permanent coding-agent operating rules.
5. The current milestone prompt for the exact authorized scope.

A status file or completion report never overrides contradictory code, tests, logs, or Git history.

1.3 Core terminology

Term	Meaning
Backtest	Historical simulation using only information available at each simulated decision point.
Out-of-sample	A chronological segment not used to select the tested candidate.
Walk-forward validation	Repeated development and out-of-sample windows across time.
Forward test	Testing on data as it arrives. Official forward testing is MOMO Quant paper trading.
Candidate	A versioned strategy definition plus parameters, provenance, and evidence.
Campaign	A bounded autonomous research objective with frozen data, policy, strategy family, and resource limits.
Hard gate	A mandatory condition that cannot be offset by a high aggregate score.
Strategy DSL	A controlled versioned schema interpreted by research and verification systems.

2. Executive Summary

Core product idea

Zorqen Research is an autonomous research laboratory, not a trading bot. It proposes controlled strategy changes, runs deterministic tests, measures results, retains only defensible improvements, and creates auditable candidate packages for independent verification by MOMO Quant.

The application is separate from MOMO Quant so aggressive research cannot destabilize the execution platform or interrupt its current development. The systems share explicit strategy semantics and integration contracts, but remain operationally isolated.

The initial product is CPU-first and must run on a Windows laptop. The same services must later run on Windows or Linux VPS infrastructure. GPU support may be added later but is not an MVP dependency.

2.1 Initial strategy scope

Priority	Strategy family	Role
Primary	Adaptive Multi-Timeframe Trend Breakout	Main autonomous research target and majority of early campaign capacity.
Secondary	Support and Resistance	Proves the engine is multi-strategy and not hardcoded around trend logic.

A campaign scheduler may default to an approximate 75/25 allocation, but this must remain configurable rather than hardcoded.

2.2 Fundamental separation of duties

Zorqen Research may	Zorqen Research must not
Generate controlled candidates	Place paper or live orders
Run historical and walk-forward tests	Modify MOMO Quant code or data
Tune approved rule blocks and parameters	Approve its own candidate for production
Reject or retain experiments within a campaign	Change frozen evaluation rules during a campaign
Export qualified candidate packages	Bypass independent verification or human approval

3. Vision, Mission and Product Principles

3.1 Vision

Build a trustworthy autonomous research organization for systematic trading strategies, where every accepted improvement is measurable, reproducible, reviewable, and separated from execution authority.

3.2 Mission

Zorqen Research reduces the manual cost of experimentation without reducing scientific discipline. It turns strategy research into a controlled loop of hypothesis, implementation, simulation, validation, rejection or retention, and documented transfer.

3.3 Product principles

Principle	Required interpretation
Evidence over claims	Reports and charts do not count without reproducible evidence.
Immutable evaluation	Data snapshots, evaluator code, and campaign policy freeze before experiments begin.
No single-metric optimization	Profit, win rate, or Sharpe alone cannot qualify a strategy.
Hard gates before scores	Fatal defects reject a candidate regardless of aggregate score.
Research/execution separation	The research app has no exchange credentials or order authority.
Determinism first	Identical inputs should reproduce identical outputs.
Controlled search first	Initial agents evolve schemas and approved rule blocks, not unrestricted code.
Human-readable auditability	Every retained change needs a plain-language rationale and machine-verifiable provenance.
Portability	Local Windows and Windows/Linux VPS operation are first-class requirements.
Fail honestly	A failed campaign is useful when failure is reproducible and documented.

4. Scope, Boundaries and Non-Goals

4.1 In scope

- Autonomous experiment campaigns for approved strategy families.
- Parameter tuning, approved rule-block changes, and controlled structural variants.
- Deterministic bar-based backtesting for Binance Futures assumptions.
- Multi-timeframe evaluation using canonical UTC candle data.
- Chronological development, out-of-sample, walk-forward, hidden-holdout, and robustness workflows.
- Realistic fees, slippage, funding, tick-size, and order-fill assumptions.
- Candidate scoring, hard rejection gates, provenance tracking, reports, and exports.
- Versioned candidate transfer to MOMO Quant through file export first and API later.

4.2 Explicit MVP non-goals

- Direct exchange connectivity or exchange API keys.
- Paper or live trade execution.
- Automatic promotion to live deployment.
- Unrestricted AI-generated Python strategy code.
- News, social sentiment, alternative-data, options, or HFT strategies.
- Cross-exchange arbitrage or distributed GPU training.
- Replacing human judgment about production risk.

4.3 Scope-control rule

Do not widen scope silently

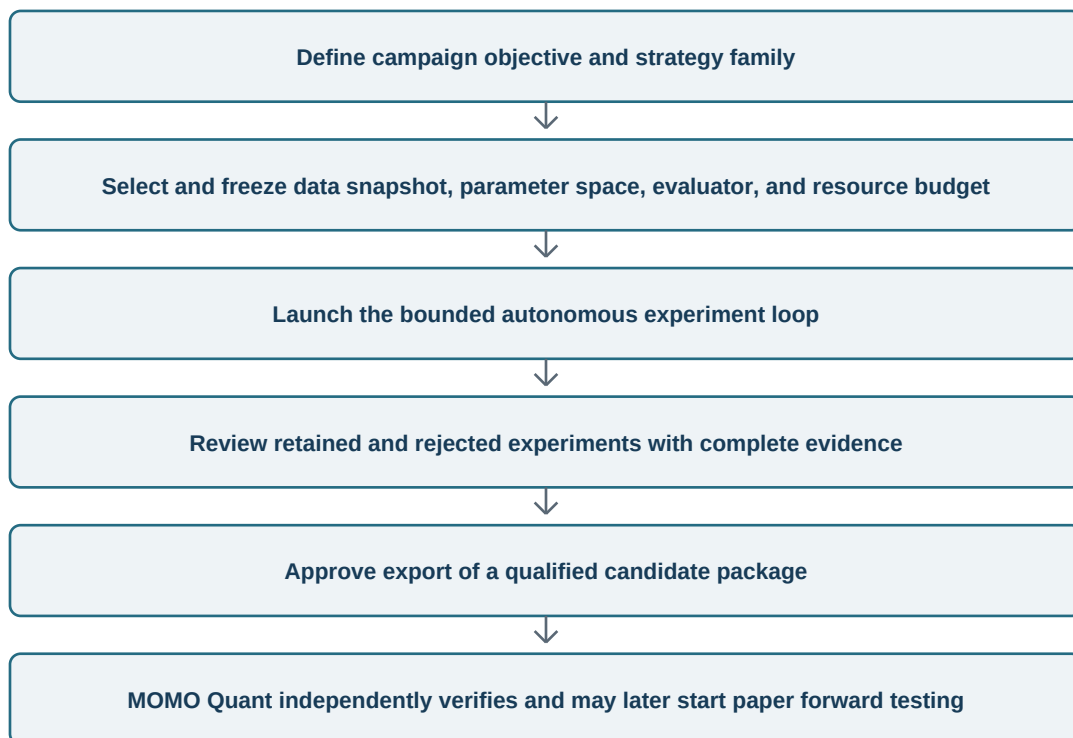
New strategy families, arbitrary code evolution, extra exchanges, machine-learning models, distributed workers, or live-data research require explicit later milestones. They are not implied by the word autonomous.

5. Roles and Operating Model

5.1 Initial roles

Role	Authority
Owner / Research Administrator	Creates campaigns, freezes policies, manages data, approves export, and controls integrations.
Research Reviewer	Inspects evidence and failures but cannot change frozen campaign inputs.
Autonomous Research Agent	Proposes and executes permitted experiments within campaign constraints.
Worker Service	Runs deterministic backtests, validation, and stress tests; makes no policy decisions.
MOMO Quant Verifier	Imports packages, reruns official verification, and controls paper forward testing.

5.2 Human workflow



5.3 Bounded autonomy

Once launched, an agent should not stop after its first implementation attempt. It continues until the campaign budget is exhausted, a configured stop condition is reached, an integrity failure occurs, or an operator cancels it.

Autonomy is constrained by permissions, immutable inputs, experiment limits, timeouts, storage limits, and policy gates. Unbounded "never stop" behavior is not acceptable in production.

6. Initial Strategy Families

6.1 Shared requirements

- Each family has a stable identity, versioned baseline, supported timeframes, required data, warm-up length, and explicit outputs.
- The research representation separates immutable semantics from tunable values.
- Every indicator and rule consumes only information available at decision time.
- A baseline must be proven against an authoritative reference before autonomous changes begin.

6.2 Adaptive Multi-Timeframe Trend Breakout - primary

This is the flagship research family. Zorqen Research must first reproduce the authoritative behavior from MOMO Quant or an approved formal specification; it must not invent a different baseline while claiming parity.

Permitted early research dimensions

- Higher-timeframe trend definition and approved trend-filter alternatives.
- Lower-timeframe breakout construction, close confirmation, and retest behavior.
- Volatility, liquidity, and multi-timeframe agreement filters.
- Stop placement, target selection, session rules, and setup invalidation.
- Long/short symmetry and regime-specific eligibility.

Protected semantics

- No future higher-timeframe candle values may leak into lower-timeframe decisions.
- Bar-close versus intrabar assumptions must be explicit and consistent.
- Account-level position sizing remains outside candidate-controlled logic.

6.3 Support and Resistance - secondary

This family proves the platform can research a structurally different strategy class. Its authoritative baseline must also be captured before autonomous experiments begin.

Permitted early research dimensions

- Zone detection, lookback, swing sensitivity, width, merging, and invalidation.
- Rejection, breakout, and breakout-retest variants.
- Volume, volatility, trend context, and false-breakout filters.
- Stop placement relative to zones and target-selection logic.

7. Controlled Strategy Representation

7.1 Why unrestricted code generation is deferred

Allowing arbitrary strategy code immediately creates hidden state, look-ahead risk, evaluator exploitation, non-determinism, security exposure, and translation problems between Python research and .NET execution. The MVP therefore uses a versioned strategy DSL or schema.

7.2 Definition layers

Layer	Contents	Change authority
Family contract	Required inputs, rule types, invariants, and output semantics	Human-controlled and versioned
Candidate definition	Selected rule blocks and parameter values	Agent within campaign policy
Execution policy	Fees, slippage, funding, and order model	Frozen campaign configuration
Account risk policy	Capital allocation and portfolio limits	MOMO Quant only
Evaluator policy	Data splits, gates, weights, and thresholds	Frozen before campaign start

7.3 Illustrative definition

```
{
  "schemaVersion": "1.0",
  "family": "adaptive_mtf_trend_breakout",
  "timeframes": {"higher": "4h", "execution": "1h"},
  "rules": {
    "trend": {"type": "ema_alignment", "fast": 50, "slow": 200},
    "breakout": {"lookback": 24, "requireClose": true},
    "retest": {"enabled": true, "toleranceAtr": 0.25, "maxBars": 6},
    "volatility": {"type": "atr_percentile", "minimum": 55}
  },
  "exit": {
    "stop": {"type": "structure_atr_buffer", "atr": 0.35},
    "target": {"type": "risk_reward", "rMultiple": 2.5}
  }
}
```

7.4 Cross-engine parity

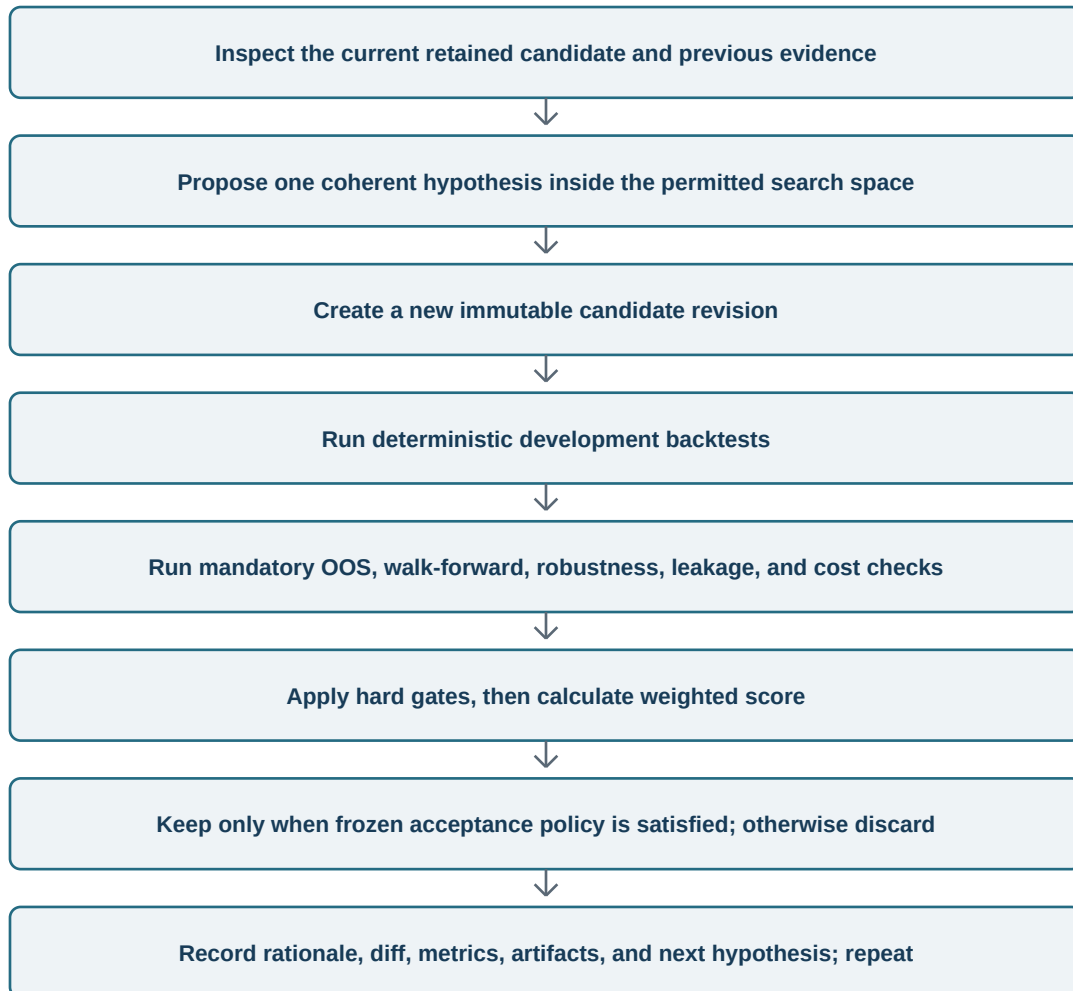
Both systems must run the same candidate against identical candles and compare signal times, direction, levels, trade lifecycle, fees, and outcomes. Parsing the same JSON is not enough; behavior must match.

8. Autonomous Research Loop

8.1 Campaign setup

Before the first experiment, the system freezes a campaign manifest containing strategy family, baseline, dataset snapshot, allowed search space, evaluator version, qualification policy, resource budget, and random seeds.

8.2 Experiment loop



8.3 Keep/discard policy

Result	Required action
Meaningful improvement	Advance the retained candidate and preserve the complete experiment record.
No meaningful change	Discard unless it materially simplifies the strategy without weakening robustness.
Regression	Discard and continue from the prior retained candidate.
Crash or invalid simulation	Record failure; fix implementation defects but do not hide an invalid idea.
Leakage or evaluator violation	Reject immediately, quarantine artifacts, and require review.

8.4 Search methods

- **Rule-aware agent proposals:** structural changes requiring domain reasoning.
- **Classical optimization:** Bayesian/TPE, random, or grid search for bounded numerical spaces.
- **Hybrid search:** the agent selects structures while an optimizer tunes numerical parameters.

9. Data Governance and Leakage Prevention

9.1 Canonical market data

- Initial exchange scope: Binance Futures only.
- Canonical timestamps: UTC.
- Initial symbols: BTCUSDT, ETHUSDT, and BNBUSDT, with later expansion by configuration.
- Target timeframes: 1m, 3m, 5m, 15m, 30m, 1h, 4h, 1d, and 1w.
- Raw data is never modified in place; corrections create a new snapshot.

9.2 Dataset snapshots

Every experiment references an immutable manifest containing source, symbols, timeframes, start/end times, row counts, gap report, normalization version, and cryptographic hashes.

9.3 Temporal isolation

- Chronological splits only; random train/test shuffling is forbidden.
- Indicator warm-up may precede a split, but performance events before the split are excluded.
- Higher-timeframe values become visible only after the higher-timeframe candle closes.
- Hidden holdout results are not exposed during iterative optimization.
- The final evaluator is inaccessible to editable candidate logic.

9.4 Data-quality gates

Check	Failure behavior
Duplicate or out-of-order candles	Reject snapshot until corrected.
Unexpected gaps	Reject or document an explicit approved gap policy.
Invalid OHLC relationships	Reject affected data.
Timezone ambiguity	Reject import.
Cross-timeframe inconsistency	Rebuild derived timeframe or reject snapshot.
Hash mismatch	Stop experiment as an integrity failure.

10. Backtesting and Validation Methodology

10.1 Simulation model

The MVP uses a deterministic event-driven bar simulator. Vectorized indicator calculations are allowed, but decisions and order lifecycle must preserve temporal order.

10.2 Required realism

- Maker/taker fees according to a frozen cost profile.
- Configurable slippage and adverse cost stress.
- Funding payments when positions cross funding timestamps.
- Tick size, quantity step, and minimum notional normalization.
- Explicit bar-close, next-bar, and limit-fill assumptions.
- Conservative deterministic handling when stop and target are both touched in the same bar.
- Long and short futures-first semantics.

10.3 Validation stages

Stage	Purpose	Agent visibility
Development	Fast iteration and initial hypothesis tests	Visible
Chronological OOS	Measure performance outside selection period	Controlled visibility
Walk-forward	Repeated adaptation across changing periods	Visible summary
Robustness	Parameter, cost, regime, and symbol stress	Visible
Hidden holdout	Final leakage-resistant qualification check	Restricted
Paper forward	Real data as it arrives	MOMO Quant only

10.4 Reproducibility

Every run records candidate hash, code commit, evaluator version, dataset manifest, configuration, seed, environment fingerprint, start/end time, metrics, and artifact locations.

11. Qualification Policy: Hard Gates and Score

11.1 Hard rejection gates

- Look-ahead bias, data leakage, evaluator tampering, or integrity failure.
- Non-deterministic results outside a documented tolerance.
- Insufficient trade sample for the configured strategy and timeframe.
- Negative OOS expectancy after realistic costs.
- Maximum drawdown above policy.
- Failure under required fee/slippage stress.
- Unacceptable profit concentration in a few outlier trades.
- Cliff-like parameter instability.
- Failure of required strategy-engine parity fixtures.

11.2 Provisional weighted score

After every hard gate passes, a 0-100 score ranks candidates. Weights are configurable but frozen per campaign.

Component	Weight	Examples
Out-of-sample quality	20	Expectancy, profit factor, and consistency after costs
Risk quality	20	Drawdown, downside deviation, and loss clustering
Walk-forward robustness	20	Positive-window rate and degradation
Parameter stability	15	Neighborhood sensitivity and plateau quality
Cross-market/regime behavior	10	Symbols and regime diversity
Execution realism	10	Cost stress, turnover, and fill sensitivity
Simplicity/auditability	5	Complexity cost, rule count, and interpretability

11.3 Initial qualification policy

A provisional candidate may become **ResearchQualified** at a score of 75 or higher only when every hard gate passes. This is not live readiness; MOMO Quant still reruns it independently.

11.4 Provisional thresholds

Measure	Initial recommendation	Status
OOS profit factor	At least 1.20	Configurable
OOS expectancy	Greater than zero after all modeled costs	Hard gate
Maximum drawdown	No more than 12-15% for the research profile	Configurable hard gate
Walk-forward positive windows	At least 60%	Configurable
Cost stress	Positive at 1.5x baseline fees/slippage	Hard gate
Profit concentration	No single trade above 20% of total net profit	Configurable
OOS degradation	No more than 35% versus development quality	Configurable
Parameter neighborhood	No cliff-like failure around selected values	Hard gate

Thresholds require calibration against authoritative baselines before production use.

12. Candidate Lifecycle



12.1 Authority limits

Zorqen Research may move a candidate only through ResearchQualified. A human authorizes export. States after import belong to MOMO Quant.

12.2 Immutability

A qualified candidate is immutable. Any logic, parameter, cost, or dataset change creates a new candidate version and requires new evidence.

12.3 Rejection as a first-class result

Rejected candidates remain searchable with their failed gates, rationale, metrics, and artifacts so the system does not repeatedly rediscover known failures.

13. Candidate Package and MOMO Quant Integration

13.1 Integration sequence

The first integration is a versioned JSON package exported by Zorqen Research and manually imported into MOMO Quant. An authenticated API comes later, after the contract and parity checks are stable.

13.2 Required contents

- Candidate identity, family, schema version, and full definition.
- Source commit, candidate hash, campaign and experiment provenance.
- Dataset manifests, evaluator version, and policy version.
- Cost assumptions and detailed stage/window metrics.
- Hard-gate outcomes, weighted score, and decision.
- Stability and stress-test evidence, known limitations, and integrity checksum.

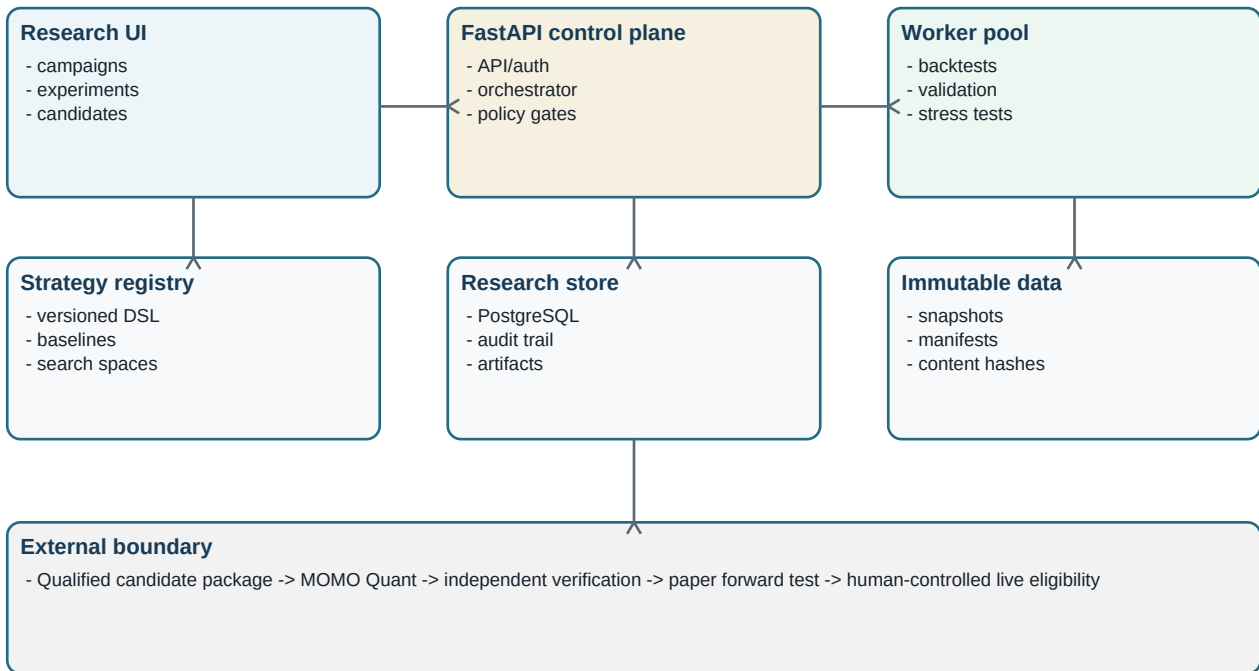
13.3 Illustrative envelope

```
{
  "packageVersion": "1.0",
  "candidateId": "cand_20260803_0042",
  "family": "adaptive_mtf_trend_breakout",
  "candidateHash": "sha256:...",
  "sourceCommit": "a73bd19",
  "definition": {"...": "..."},
  "datasetManifests": ["dataset:binance-futures-v3"],
  "evaluatorVersion": "evaluator-1.2.0",
  "qualificationPolicy": "policy-2026-08-a",
  "decision": {
    "status": "ResearchQualified",
    "score": 79.2,
    "hardGateFailures": []
  }
}
```

13.4 MOMO Quant responsibilities

- Validate package schema and integrity.
- Map the strategy definition to the official execution interpreter.
- Rerun the candidate on authoritative data and evaluator code.
- Run cross-engine parity comparison and reject unexplained differences.
- Control paper forward testing, account risk, and live eligibility.

14. System Architecture



14.1 Major components

Component	Responsibility
Web UI	Campaign creation, monitoring, comparison, review, policy, and data administration.
FastAPI control plane	Authentication, APIs, orchestration, policy enforcement, and lifecycle transitions.
Campaign orchestrator	Creates bounded work, tracks the retained baseline, and enforces stop conditions.
Research-agent adapter	Produces structured hypotheses and candidate modifications within permitted tools.
Worker service	Runs backtests, validation, robustness checks, and artifact generation.
Strategy registry	Stores family contracts, schemas, baselines, search spaces, and interpreters.
Data service	Imports, validates, snapshots, and serves immutable candle data.
Evaluator	Applies metrics, hard gates, and frozen scoring policy.
Candidate registry	Stores immutable revisions and lifecycle state.
Artifact store	Persists logs, reports, charts, trades, and reproducibility bundles.
Integration gateway	Exports packages and later calls MOMO Quant APIs.

14.2 Initial deployment shape

Use a modular monolith with separate API and worker processes. Premature microservices would add operational cost without improving research correctness.

15. Recommended Technology Stack

Area	Initial recommendation	Reason
Backend/API	Python 3.12, FastAPI, Pydantic	Typed contracts and strong numerical ecosystem.
Research engine	NumPy plus pandas/Polars where appropriate; deterministic event engine	Efficient analysis while preserving temporal semantics.
Database	PostgreSQL plus Alembic	JSONB, transactions, leasing, and portable migrations.
Worker model	Dedicated Python worker with database-backed leasing	Cross-platform without premature queue infrastructure.
Frontend	React, Vite, TypeScript	Strong dashboard tooling and familiar stack.
Testing	pytest, Hypothesis, Vitest, Playwright	Unit, property, integration, and UI coverage.
Quality	Ruff, type checking, pre-commit	Fast automated quality gates.
Packaging	Docker Compose plus direct Windows scripts	Local development and VPS portability.
Artifacts	Filesystem abstraction; object storage later	Simple MVP without blocking scale.
Data format	Parquet for large tables; JSON for contracts	Efficient storage and readable interfaces.

15.1 CPU-first rule

The initial engine must not require CUDA. GPU dependencies may be optional later. Parallelism should be introduced only after deterministic single-worker behavior is proven.

15.2 Windows/Linux parity

- No hardcoded drive paths in core code.
- Use pathlib and configuration-based storage roots.
- Avoid shell-only orchestration assumptions.
- Provide PowerShell and POSIX-friendly commands where practical.
- Run core CI tests on Windows and Linux before VPS deployment.

16. Core Domain Model

Entity	Purpose
StrategyFamily	Stable family identity, contract version, and capabilities.
StrategyBaseline	Authoritative starting definition and parity evidence.
Candidate	Immutable strategy revision and content hash.
Campaign	Frozen objective, budgets, policy, and retained candidate.
Experiment	One hypothesis, candidate diff, execution result, and decision.
DatasetSnapshot	Immutable manifest and quality evidence.
EvaluationPolicy	Versioned metrics, hard gates, weights, and thresholds.
BacktestRun	Deterministic simulation inputs, state, and outputs.
ValidationRun	OOS, walk-forward, robustness, or hidden-holdout result.
QualificationDecision	Gate outcomes, score, and status.
Artifact	Addressed log, report, chart, trades, or reproducibility bundle.
CandidatePackage	Export envelope and integrity checksum.
WorkerLease	Transactional ownership and heartbeat for queued work.
AuditEvent	Append-only lifecycle and security event.

16.1 State machines and idempotency

Lifecycle transitions are enforced by domain services rather than arbitrary database updates. Every transition records actor, time, reason, and evidence. Repeated commands use idempotency keys so duplicate requests do not create duplicate runs or candidates.

17. API and User Interface

17.1 Initial API groups

Route group	Responsibilities
/api/v1/strategy-families	Contracts, baselines, schemas, and search spaces.
/api/v1/datasets	Import, validate, snapshot, and inspect data.
/api/v1/policies	Create and version evaluation policies.
/api/v1/campaigns	Create, launch, pause, cancel, and inspect campaigns.
/api/v1/experiments	Hypotheses, diffs, status, logs, and results.
/api/v1/candidates	Compare, qualify, reject, and export candidates.
/api/v1/workers	Health, leases, capacity, and diagnostics.
/api/v1/artifacts	Artifact metadata and secure downloads.
/api/v1/integrations/momo-quant	Package validation, export, and later transfer.

17.2 Required MVP screens

Screen	Primary content
Dashboard	Workers, active campaigns, recent decisions, failures, and qualified candidates.
Strategy Families	Baselines, schemas, capabilities, and parity status.
Datasets	Snapshots, symbols, timeframes, quality checks, and hashes.
Campaign Builder	Objective, family, search space, data, policy, budgets, and stop conditions.
Campaign Detail	Progress, retained candidate, funnel, resource use, and alerts.
Experiment Detail	Hypothesis, diff, logs, metrics, gates, charts, and decision.
Candidate Comparison	Side-by-side metrics, stability, windows, and complexity.
Candidate Detail	Provenance, evidence, limitations, and export controls.
Operations	Workers, queues, failures, storage, audit, and environment.

17.3 UX principles

- Do not use raw JSON as the primary experience.
- Clearly separate development, OOS, walk-forward, hidden-holdout, and paper-forward results.
- Show why a candidate failed and display sample size beside performance metrics.
- Never present ResearchQualified as ready for live trading.

18. Security, Isolation and Governance

18.1 Security boundaries

- No exchange trading credentials are stored.
- Research-agent tools are allowlisted by role and campaign.
- MVP candidate definitions cannot invoke arbitrary operating-system commands.
- Workers use restricted filesystem access and resource limits.
- Export packages are integrity-checked; later API transfer uses authenticated service credentials.
- Secrets are provided through environment or secret stores, never committed.

18.2 Governance

- Only humans approve candidate export.
- Only MOMO Quant starts paper forward testing.
- Only a human-controlled workflow grants live eligibility.
- Policy changes create new versions and never rewrite historical decisions.

18.3 Future arbitrary-code sandbox

If unrestricted code evolution is introduced later, it must run in isolated containers with no network by default, read-only data mounts, CPU/memory/time limits, ephemeral filesystems, syscall restrictions, and signed output manifests. This is outside the MVP.

19. Observability, Failure Handling and Retention

19.1 Required telemetry

- Structured logs with campaign, experiment, candidate, run, and worker identifiers.
- Queue depth, lease age, runtime, throughput, failure rate, memory, and CPU metrics.
- Health endpoints for API, database, artifact store, and workers.
- Progress events for live UI updates and detection of stuck or orphaned work.

19.2 Failure handling

Failure	Expected behavior
Worker crash	Lease expires and work is safely retried under idempotent policy.
Backtest timeout	Mark failed, retain logs, and continue only when policy permits.
Artifact write failure	Do not mark run complete; retry or fail atomically.
Database interruption	Stop new work and avoid duplicate transitions.
Invalid candidate	Reject before expensive execution.
Repeated infrastructure failure	Pause campaign and require operator review.
Integrity mismatch	Stop affected work and raise a high-severity alert.

19.3 Retention

Campaign manifests, candidate definitions, decisions, and summary metrics remain durable. Large intermediate artifacts may use configurable retention, but evidence supporting a qualified export must remain available.

20. Testing and Quality Gates

20.1 Test layers

Layer	Examples
Unit	Indicators, rules, metrics, scores, and lifecycle guards.
Property-based	OHLC invariants, no-future-data behavior, idempotency, and bounds.
Golden master	Known candle fixtures with exact signals, trades, and metrics.
Integration	Transactions, worker leasing, artifact writes, and APIs.
Parity	Zorqen Research versus MOMO Quant equivalence.
Determinism	Repeated runs produce identical output hashes.
Leakage	Tests attempt future access and evaluator manipulation.
Performance	Representative runtime and memory budgets.
Frontend	Components, workflows, and browser tests.
Deployment	Windows/Linux container and migration smoke tests.

20.2 Milestone completion gates

- Every acceptance criterion is individually audited.
- Targeted tests pass, followed by required broader suites.
- Migrations work from a clean database and the previous verified schema.
- Logs are inspected for hidden exceptions and warnings.
- The final diff contains only authorized scope.
- AGENTS.md and PROJECT_STATUS.md remain accurate.
- The milestone commit is created only after verification.

Completion rule

A coding agent continues the implement-test-inspect-correct loop until milestone gates pass. A successful build alone is not completion, and a completion summary is not evidence.

21. Repository Documentation and Agent Handoff

21.1 Required root files

File	Purpose
README.md	Setup, local run, architecture overview, and contributor entry point.
AGENTS.md	Permanent working rules for every coding agent.
PROJECT_STATUS.md	Current verified state, progress, problems, and next authorized work.
docs/contracts/	Versioned schemas and integration examples.
docs/adr/	Architecture decision records requiring durable rationale.

21.2 New-agent read order

1. README.md.
2. AGENTS.md.
3. PROJECT_STATUS.md.
4. Current milestone prompt.
5. Git status, branch, recent log, and diff from last verified commit.
6. Relevant tests before implementation files.
7. Relevant architecture decisions and contracts.

21.3 Agent switching

A new coding agent must independently verify repository state. PROJECT_STATUS.md is an orientation aid, not proof. This preserves continuity without inheriting unverified claims.

22. Deployment Model

22.1 Local development

- Windows laptop as the primary environment.
- Direct commands for API, worker, and frontend.
- Docker Compose for PostgreSQL and optional full-stack runs.
- Configurable storage roots and no fixed drive letters.
- Single worker by default for deterministic debugging.

22.2 VPS deployment

- Windows or Linux supported at the architecture level.
- Linux containers are preferred for production when practical.
- Separate API, worker, and database processes.
- Reverse proxy, TLS, backups, and monitored storage.
- Worker concurrency based on measured CPU and memory.

22.3 Scaling sequence

1. Prove one deterministic local worker.
2. Add safe parallel workers on one machine.
3. Add remote worker registration and leasing.
4. Introduce object storage and dedicated queue infrastructure only when measured demand requires them.

23. Delivery Roadmap and Milestones

The numbering below defines dependency order. It is not permission to implement all phases at once.

Phase	Objective	Exit evidence
0 - Foundation	Repository, stack, CI, configuration, health, AGENTS.md, and PROJECT_STATUS.md	Clean builds, tests, and cross-platform setup
1 - Core domain	Entities, state machines, database, audit, and migrations	Transactional lifecycle and migration tests
2 - Data foundation	Import, validation, snapshots, hashes, and timeframes	Immutable manifests and quality tests
3 - Simulator	Order model, fees, slippage, funding, metrics, and fixtures	Deterministic output hashes and leakage tests
4 - Strategy schema	Versioned DSL and baselines for both strategies	Schema validation and baseline parity
5 - Experiment engine	Campaigns, loop, revisions, and keep/discard	Bounded autonomous campaign completes reproducibly
6 - Validation	OOS, walk-forward, robustness, gates, and scoring	Qualification decisions with evidence
7 - Research UI	Campaign, experiment, candidate, data, and policy workflows	End-to-end browser-tested workflow
8 - Candidate packaging	Versioned export, integrity, and reports	Package validates and reruns independently
9 - MOMO integration	Manual import first; API later; parity and handoff	Independent MOMO verification path
10 - Operations	Security, recovery, backups, retention, and VPS	Production-readiness checklist
11 - Advanced autonomy	Hybrid optimization, remote workers, optional sandbox	Only after MVP evidence

23.1 Milestone prompt structure

Every coding-agent prompt must name the authoritative base commit, exact scope, exclusions, acceptance criteria, required tests, loop-based procedure, documentation updates, commit rules, and final evidence format.

23.2 No milestone overlap

A new milestone does not start until the previous commit is independently reviewed. Corrective work receives an explicit corrective milestone rather than being hidden inside later scope.

24. MVP Acceptance Criteria

24.1 Product acceptance

- Verified baselines can be registered for both initial strategy families.
- An administrator can create a campaign with frozen data, policy, search space, and resource budget.
- The campaign autonomously executes multiple candidate experiments.
- The system correctly retains, discards, and records decisions.
- OOS, walk-forward, stability, and cost-stress stages execute.
- Hard-gate failures cannot be overridden by score.
- A qualifying candidate produces a complete immutable package.
- A reviewer can reconstruct the chain from package to campaign, experiment, candidate, data, and evaluator.

24.2 Technical acceptance

- The full stack runs on the Windows laptop without a GPU.
- Core tests run on Windows and Linux.
- Repeated deterministic runs match output hashes.
- Worker retry does not duplicate completed work.
- Database and artifact state stay transactionally consistent.
- No component contains exchange credentials or live-order capability.
- Repository handoff documents match verified reality.

24.3 Research-integrity acceptance

- Future-data access tests fail closed.
- Hidden holdout inputs are inaccessible to candidate generation.
- Policy and data changes create new versions.
- Each result reports sample size, costs, drawdown, and stage-specific metrics.
- Qualification is reproducible from stored evidence.

25. Risks and Mitigations

Risk	Consequence	Primary mitigation
Backtest overfitting	Historical results fail forward	Hidden holdout, walk-forward, stability, and simplicity penalties
Metric gaming	Agent exploits evaluator weaknesses	Immutable evaluator, inaccessible holdout, and leakage tests
Cross-engine mismatch	Research result cannot execute identically	Versioned DSL and mandatory parity fixtures
Unrealistic fills	Inflated performance	Explicit conservative order model and cost stress
Insufficient samples	Unreliable statistics	Hard minimum trade and window requirements
Resource runaway	Laptop/VPS exhaustion	Campaign budgets, timeouts, and worker limits
Documentation drift	Agent handoff failure	Required PROJECT_STATUS.md update and verification
Scope expansion	Delayed core product	Explicit non-goals and milestone authorization
Data corruption	Invalid research	Immutable snapshots, hashes, and quality gates
Premature live trust	Financial loss	Research/execution separation and human lifecycle control

25.1 Primary product risk

The largest risk is not failure to find a good strategy. It is producing convincing but invalid evidence. Architecture and testing must optimize first for research integrity, then for experiment throughput.

26. Decisions Frozen by Version 0.1

Decision	Status
Umbrella brand	Zorqen
New application name	Zorqen Research
Execution platform name	MOMO Quant for now; possible future Zorqen Quant
Product separation	Separate repository, database, and deployment
Initial strategies	Adaptive MTF Trend Breakout primary; Support and Resistance secondary
Initial autonomy	Controlled strategy definitions and bounded experiments
Trading authority	None
Official forward testing	MOMO Quant paper trading
Initial hardware	Windows laptop, CPU-first
Future hosting	Windows or Linux VPS
Documentation model	One master PDF plus AGENTS.md and PROJECT_STATUS.md
Development method	Milestone-based, loop-prompted agents, independent review
Integration sequence	Candidate package file first; API later

26.1 Deferred details

- Exact authoritative baseline logic and parity fixtures for both strategies.
- Final qualification thresholds after baseline calibration.
- AI model/provider for structural hypothesis generation.
- Final authentication and deployment topology.
- Final MOMO Quant candidate-import endpoint.

These must be resolved through evidence-based milestones, not silently assumed by the first coding agent.

27. Initial Repository Deliverable

The first coding milestone creates only a trustworthy foundation. It must not attempt the backtester, autonomous agent, or MOMO integration prematurely.

27.1 Expected foundation

- Repository structure for API, worker, frontend, tests, and documentation.
- Pinned toolchain and reproducible setup.
- Basic FastAPI health endpoint and React health page.
- PostgreSQL connectivity and Alembic migration framework.
- Structured configuration and logging.
- CI checks for backend and frontend.
- Root AGENTS.md and PROJECT_STATUS.md populated with verified information.
- Contract and ADR directories without speculative implementation.

27.2 Explicit exclusions

- No strategy logic.
- No backtesting engine.
- No autonomous agent integration.
- No candidate qualification.
- No MOMO Quant API calls.
- No paper or live trading.

27.3 First-commit review

After the coding agent creates the first commit, the repository will be independently inspected. The next prompt will be based on the actual code, diff, tests, and status file rather than the agent completion summary.

Appendix A. Development-Agent Loop

LOOP UNTIL ALL MILESTONE GATES PASS:

1. Inspect repository, tests, current status, and authorized scope.
2. Plan the smallest coherent implementation.
3. Implement one verified slice.
4. Run targeted tests and inspect logs.
5. Correct every failure and incomplete criterion.
6. Run broader required verification.
7. Audit the final diff against exclusions and architecture rules.
8. Update PROJECT_STATUS.md with actual commands and results.
9. Create the milestone commit only after verification.
10. Report evidence and stop. Do not begin the next milestone.

Appendix B. Research-Agent Loop

LOOP WHILE CAMPAIGN BUDGET REMAINS:

1. Read the frozen campaign manifest and current retained candidate.
2. Review previous experiments to avoid repetition.
3. Propose one permitted, testable hypothesis.
4. Create a new immutable candidate revision.
5. Execute development and mandatory validation stages.
6. Apply hard gates.
7. Calculate score only when all gates pass.
8. Keep or discard according to frozen policy.
9. Store rationale, diff, results, artifacts, and next hypothesis.
10. Stop only on budget, cancellation, terminal policy, or integrity failure.

Appendix C. Glossary

Term	Definition
Adaptive MTF Trend Breakout	Primary strategy family using higher/lower timeframe trend and breakout context.
Support and Resistance	Secondary family based on structural zones and reactions or breakouts.
Candidate hash	Content-derived identifier covering the immutable strategy definition.
Campaign manifest	Frozen configuration controlling an autonomous research campaign.
Complexity penalty	Pressure against unnecessary rules and fragile logic.
Hidden holdout	Final chronological data withheld from iterative selection.
Parity fixture	Known input comparing two interpreters for behavioral equivalence.
ResearchQualified	Passed Zorqen policy; not approved for paper or live trading.
Worker lease	Time-bounded transactional ownership of queued work.
Zorqen Candidate Package	Versioned bundle containing definition, evidence, and integrity metadata.